

NOTES

Classes and Objects



Classes and Objects

Classes and Objects in Object-Oriented Programming

What is a Class?

A class can be thought of as a blueprint or template for creating objects. It defines the structure and behaviour that the objects created from it will possess. This includes:

Attributes: These are the data members or properties that define the characteristics of the class. For example, in a **Dog** class, attributes might include:

breed: The breed of the dog (e.g., Golden Retriever, Bulldog).

colour: The colour of the dog's fur (e.g., Golden, White).

Methods: These are functions defined within the class that describe the behaviours or actions that the objects can perform. For example, a **Dog** class might have methods like:

bark(): A method that simulates the dog barking.

Here's a simple representation of a **Dog** class in Python

Class Definition:

```
Python
class Dog:
    def __init__(self, breed, color):
        self.breed = breed
        self.color = color

    def bark(self):
        print(f"{self.breed} is barking!")

dog1 = Dog("Golden Retriever", "Golden")
dog2 = Dog("Bulldog", "White")
```

Components of the Dog Class

1. **Class Declaration:** The line `class Dog:` defines a new class named `Dog`. This acts as a blueprint for creating dog objects.
2. **Constructor Method (`__init__`):** The `__init__` method is a special method called a constructor. It initialises the attributes of the class when a new object is created.

In this case, it takes two parameters: `breed` and `colour`, which are used to set the corresponding attributes for each dog object.

3. **Attributes:** `self.breed` and `self.color` are instance attributes. The `self` keyword refers to the instance of the class being created. Each dog object will have its own unique values for these attributes.
4. **Method:** The `bark()` method defines behaviour for the Dog class. When called, it prints a message indicating that the dog is barking, including its breed.

What is an Object?

An object is an instance of a class. When a class is defined, no memory is allocated until an object is created from that class. Each object can hold its own unique data for the attributes defined in its class.

For example, if you have a `Dog` class, you can create multiple objects (instances) of that class:

Creating Objects

```
Python
dog1 = Dog("Golden Retriever", "Golden")
dog2 = Dog("Bulldog", "White")
```

Object Instantiation

1. **Creating Instances:** Here, we create two instances (objects) of the `Dog` class: `dog1` is an instance representing a "Golden Retriever" with the colour "Golden".
2. **Memory Allocation:** When we instantiate `dog1` and `dog2`, memory is allocated for each object, and their respective attributes (`breed` and `colour`) are initialised with the values provided.

Attributes and Methods in Action (Calling Methods)

```
Python
dog1.bark() # Output: Golden Retriever is barking!
dog2.bark() # Output: Bulldog is barking!
```

Method Invocation

1. **Calling the bark Method:** When we call `dog1.bark()`, Python executes the `bark()` method for the `dog1` instance. The method prints: "Golden Retriever is barking!", using the value of `breed` from the `dog1` object.
2. **Output for dog2:** Similarly, calling `dog2.bark()` executes the same method but for the `dog2` instance. It prints: "Bulldog is barking!", reflecting that this object's breed is different from that of `dog1`.

Why Classes and Objects?

Classes and objects allow for organising code logically and modularly. By using classes, you can model real-world entities and group related functionalities. Objects created from these classes encapsulate the behaviour and state, allowing for code reuse. For instance, once you define a **Car** class, you can create multiple **Car** objects, each with its own unique attributes (like brand and model).

- **Organise Code:** Instead of writing all code in one long sequence, OOP structures code into reusable components.
- **Promote Reusability:** You can reuse classes across different parts of the program.
- **Scalability:** As programs grow in complexity, OOP makes it easier to manage by keeping code modular.

Example:

```
Python
class Car:
    def __init__(self, brand, model, year):
        self.brand = brand # Attribute
        self.model = model # Attribute
        self.year = year # Attribute

    def start(self): # Method
        print(f"{self.brand} {self.model} ({self.year}) is starting.")

    def stop(self): # Method
        print(f"{self.brand} {self.model} is stopping.")

# Creating two objects of the Car class
```

```
my_car = Car("Toyota", "Corolla", 2020)
another_car = Car("Honda", "Civic", 2018)

# Using object methods
my_car.start() # Output: Toyota Corolla (2020) is starting.
another_car.stop() # Output: Honda Civic is stopping.
```

In this example, the `Car` class defines attributes like `brand`, `model`, and `year`, which are initialised when an object is created. The class also has two methods, `start()` and `stop()`, which describe the behaviour of the car. Two objects, `my_car` and `another_car`, are created from the `Car` class, and each can use the methods to simulate the actions of starting and stopping.

Key Differences Between Classes and Objects

Aspect	Class	Object
Definition	A blueprint or template for creating objects. It defines the structure (attributes and methods) that its instances will have.	A specific instance of a class that contains actual values for the attributes defined by the class.
Memory Allocation	No memory is allocated when defining a class. Memory is only used when an object is instantiated.	Memory is allocated when an object is created. Each object holds space for its data.
Usage	Used to define properties (attributes) and behaviours (methods) shared by all instances.	Represents a real-world entity or specific instance, having unique values for its attributes.
Example	class Car: colour, speed, brand are attributes.	my_car = Car() my_car.color = "Red", my_car.speed = 100 are specific to the object.
Access	Acts as a model to access class methods and attributes without holding specific data.	Holds actual data and interacts with class methods to perform actions.

Creation	Defined once, acts as a template.	Multiple objects (instances) can be created from the same class.
-----------------	-----------------------------------	--

Class Components

Class Attributes(Static Attributes)

What are Class Attributes?

Class attributes are variables that are shared across all instances of a class. This means that all objects of the class will have the same value for a class attribute unless it is explicitly changed. These attributes are defined directly within the class but outside any method, making them accessible to all objects created from the class.

Why use Class Attributes?

Class attributes are ideal when you want to define a characteristic that should be the same for all instances of a class. For example, if you were creating a class for cars, you might want all cars to have the same number of wheels. In this case, the number of wheels can be defined as a class attribute, as it doesn't change from one object to another.

How to define Class Attributes?

Class attributes are defined directly inside the class, but outside of any methods. They are shared among all instances of the class and can be accessed using either the class name or an instance.

Example: In a Car class, wheels could be a class attribute, assuming all cars have four wheels.

Python

```
class Car:  
    wheels = 4 # Class attribute
```

Instance Attributes

What are Instance Attributes?

Instance attributes, on the other hand, are specific to each object (instance) created from a class. They are typically defined in the `__init__()` method (the constructor) and can have different values for each object.

Why use Instance Attributes?

Instance attributes allow each object to have its own unique data. For example, while all cars may have the same number of wheels, each car will have its own brand or model, which is unique to that particular car. Instance attributes provide this flexibility, allowing each object to store its own individual state.

How to define Instance Attributes?

Instance attributes are usually defined inside the `__init__()` method, and the keyword `self` is used to refer to the object itself. When an object is created, these attributes are initialised with the values provided.

Example: make and model in the Car class are instance attributes since they vary per car.

Example: Class Attributes vs. Instance Attributes

```

Python
class Car:
    wheels = 4 # Class attribute (shared by all instances)

    def __init__(self, brand, model):
        self.brand = brand # Instance attribute (specific to each
object)
        self.model = model # Instance attribute

# Creating objects (instances) of the Car class
car1 = Car("Toyota", "Corolla")
car2 = Car("Honda", "Civic")

# Accessing class attribute
print(car1.wheels) # Output: 4
print(car2.wheels) # Output: 4

# Accessing instance attributes
print(car1.brand, car1.model) # Output: Toyota Corolla
print(car2.brand, car2.model) # Output: Honda Civic
  
```

In this example:

- `wheels` is a class attribute that is shared across all instances of the `Car` class. Both `car1` and `car2` have 4 wheels because the class attribute is the same for all cars.
- `brand` and `model` are instance attributes. Even though `car1` and `car2` share the same class, their brands and models are different because these attributes are specific to each object and initialised individually in the `__init__()` method.

Changing Class and Instance Attributes:

- Changing a class attribute will affect all instances that don't override it.
- Changing an instance attribute only affects the specific object.

Python

```
car1.wheels = 3 # Changing the class attribute for car1 instance
print(car1.wheels) # Output: 3
print(car2.wheels) # Output: 4 (unaffected by the change in car1)

car2.model = "Accord" # Changing instance attribute for car2
print(car1.model) # Output: Corolla (unaffected by the change in car2)
print(car2.model) # Output: Accord
```

Here, changing the `wheels` class attribute for `car1` only affects `car1` and not `car2`. Likewise, changing `car2`'s `model` attribute doesn't affect `car1`. This showcases the independence of instance attributes and the shared nature of class attributes.

Methods

What?

Methods are functions defined inside a class that operate on the object's attributes. They define the behaviours of the object and how it interacts with its data. A method is similar to a regular function but belongs to an instance of the class and can access its attributes.

Why?

Methods allow objects to perform actions and interact with their attributes in meaningful ways. By using methods, we encapsulate the logic and behaviours inside the object itself, promoting code organisation, reuse, and modularity. Methods define how objects behave and interact with each other, making them essential for object-oriented programming.

How?

To define a method, you use the `def` keyword inside the class. The first parameter must always be `self`, which refers to the instance of the object (i.e., the current object). When calling the method, the `self` parameter is automatically passed, so you don't need to provide it explicitly.

Example 1:

```
Python
class Person:
    def __init__(self, name):
        self.name = name

    def greet(self):
        print(f"Hello, my name is {self.name}")

person = Person("Alice")
person.greet() # Output: Hello, my name is Alice
```

The `greet` method is defined to print a greeting using the `name` attribute. The `self` parameter allows the method to access the `name` attribute, specific to each instance of the `Person` class. When `person.greet()` is called, it prints "Hello, my name is Alice."

Example 2:

```
Python
class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2

circle1 = Circle(5)
print(circle1.area()) # Output: 78.5
```

The `Circle` class has a method `area()` that calculates the area of the circle using the `radius` attribute. Each instance of `Circle` can calculate its area independently. The `self.radius` accesses the radius specific to that particular circle instance.

The `__init__()` Method (Constructor)

What is the `__init__()` Method?

The `__init__()` method, also called a **constructor**, is responsible for initialising the newly created object's state. It sets up the initial values for the object's attributes (usually instance attributes) when the object is instantiated. This method is automatically called when an object is created, which eliminates the need for the user to manually initialise the attributes after creating the object.

Why use the `__init__()` Method?

The `__init__()` method is essential because it simplifies the object creation process. Without this method, attributes would have to be set manually after the object is created. By using the `__init__()` method, you can ensure that all necessary attributes are properly initialised when the object is created, avoiding potential errors or uninitialized data.

How to define the `__init__()` Method?

The `__init__()` method is defined inside the class, and it takes `self` as the first argument, which refers to the instance of the class. Other arguments are used to initialise the object's attributes.

Example:

```
Python
class Car:
    def __init__(self, brand, model, year): # Constructor
        self.brand = brand # Instance attribute
```

```
self.model = model # Instance attribute
self.year = year # Instance attribute

def display_info(self):
    print(f"Car: {self.brand} {self.model} ({self.year})")

# Creating an object of the Car class
my_car = Car("Tesla", "Model S", 2022)

# Accessing instance attributes and methods
my_car.display_info() # Output: Car: Tesla Model S (2022)
```

In this example, the `__init__()` method initialises three instance attributes—`brand`, `model`, and `year`—when a `Car` object is created. The `my_car` object is created with specific values for each attribute, and we can call the `display_info()` method to display the car's information. The `__init__()` method ensures that every `Car` object has its attributes properly set when it is created.

Multiple Objects with Different Attributes

```
Python
car1 = Car("BMW", "X5", 2020)
car2 = Car("Audi", "Q7", 2021)

car1.display_info() # Output: Car: BMW X5 (2020)
car2.display_info() # Output: Car: Audi Q7 (2021)
```

In this case, two objects `car1` and `car2` are created from the `Car` class, each with its own `brand`, `model`, and `year`. The `__init__()` method ensures that these attributes are unique to each object, allowing us to create multiple cars with different properties.

The self Parameter

The `self` parameter is an essential part of working with classes in Python. It's the way objects in Python can keep track of their own data and behaviour. Here's a breakdown of what `self` is, why it's used, and how it enables each object (or instance) to maintain its own unique state.

What is self?

In Python, `self` represents the instance of the class. When a method is called on an object, `self` is automatically passed as the first argument. By convention, it's named `self`, but you could technically name it anything (though it's strongly recommended to stick to `self` for readability and consistency).

Why Use self?

- **Allows Object-Specific Data:** Each object can have unique data (attributes), and `self` helps keep track of each object's attributes.
- **Accessing Methods:** It allows objects to call their own methods and update their own attributes.
- **Consistency:** It makes code readable and structured by keeping data and functions that operate on that data together within each object.

Example 1:

```
Python
class Car:
    def __init__(self, make, model, year):
        self.make = make          # self.make refers to the instance's
        make attribute            # self.model refers to the instance's
        self.model = model        # self.year refers to the instance's
        model attribute           year attribute
        self.year = year

    def car_info(self):
        # Using self to access instance attributes
        return f"{self.year} {self.make} {self.model}"

    def start_engine(self):
        # Method that uses instance attributes
        print(f"The engine of {self.make} {self.model} is now
        running.")

# Creating instances of Car
car1 = Car("Toyota", "Corolla", 2020)
car2 = Car("Honda", "Civic", 2021)

# Each instance has its unique data
print(car1.car_info()) # Output: 2020 Toyota Corolla
print(car2.car_info()) # Output: 2021 Honda Civic
# Calling start_engine method on each car
car1.start_engine() # Output: The engine of Toyota Corolla is now
running.
car2.start_engine() # Output: The engine of Honda Civic is now
running.
```

In this example:

- The `__init__` method uses `self` to initialise `make`, `model`, and `year` attributes for each `Car` object.
- `car_info()` and `start_engine()` use `self` to access each car's individual attributes.

Example 2:

```
Python
class Account:
    def __init__(self, owner, balance=0):
        self.owner = owner          # Account holder's name
        self.balance = balance      # Initial balance for the account

    def deposit(self, amount):
        self.balance += amount      # Increase balance by deposit
        amount
        print(f"Deposited {amount}. New balance is {self.balance}.")

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount  # Decrease balance by withdrawal
            amount
            print(f"Withdrew {amount}. New balance is {self.balance}.")
        else:
            print("Insufficient funds!")

# Creating two separate Account objects
account1 = Account("Alice", 100)
account2 = Account("Bob", 50)

# Each account maintains its own balance
account1.deposit(30) # Output: Deposited 30. New balance is 130.
account2.deposit(20) # Output: Deposited 20. New balance is 70.
account1.withdraw(50) # Output: Withdrew 50. New balance is 80.
account2.withdraw(100) # Output: Insufficient funds!
```

In this example:

- `self.owner` and `self.balance` track each account's individual owner and balance.
- The `deposit` and `withdraw` methods use `self` to modify each account's specific balance.

Accessing and Modifying Object Attributes

Once an object is created, its attributes can be accessed or modified directly using dot notation. This is useful when you want to retrieve or update the properties of an object after its creation. Object attributes store information about the object's state, and modifying them allows you to update or change that state throughout the lifecycle of the object. You access an attribute using `object.attribute`. To modify an attribute, simply assign a new value using the same dot notation.

Example 1:

```
Python
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

my_car = Car("Toyota", "Corolla")
print(my_car.brand) # Accessing the attribute: Output -> Toyota

my_car.model = "Camry" # Modifying the attribute
print(my_car.model) # Output -> Camry
```

After creating the `my_car` object with the brand "Toyota" and model "Corolla," you can easily access and modify these attributes using dot notation. In this case, the model of the car is updated from "Corolla" to "Camry."

Example 2:

```
Python
class Laptop:
    def __init__(self, brand, price):
        self.brand = brand
        self.price = price

laptop1 = Laptop("Dell", 1200)
print(laptop1.price) # Accessing the attribute: Output -> 1200

laptop1.price = 1000 # Modifying the attribute
print(laptop1.price) # Output -> 1000
```

Building and Instantiating Classes with Attributes

A class can have multiple attributes, and when instantiating the class, you can pass values to initialise those attributes. Defining multiple attributes in a class allows you to create objects with unique values for each instance. This gives you the flexibility to model real-world entities with multiple properties. Define multiple attributes inside the `__init__` method of the class. When creating an object, pass the appropriate values for those attributes.

Example 1:

```
Python
class Smartphone:
    def __init__(self, brand, model, price):
        self.brand = brand
        self.model = model
        self.price = price

phone1 = Smartphone("Apple", "iPhone 12", 999)
```

```
phone2 = Smartphone("Samsung", "Galaxy S21", 799)

print(phone1.brand, phone1.price) # Output: Apple 999
print(phone2.brand, phone2.price) # Output: Samsung 799
```

In this example, we define three attributes (**brand**, **model**, and **price**) for the **Smartphone** class. The **phone1** and **phone2** objects are created with different values for each of these attributes, illustrating how each object instance can have its own unique properties while still being of the same class.

Example 2:

```
Python
class Book:
    def __init__(self, title, author, pages):
        self.title = title
        self.author = author
        self.pages = pages

book1 = Book("1984", "George Orwell", 328)
book2 = Book("To Kill a Mockingbird", "Harper Lee", 281)

print(book1.title, book1.pages) # Output: 1984 328
print(book2.title, book2.pages) # Output: To Kill a Mockingbird 281
```

The **Book** class models a real-world book with attributes such as **title**, **author**, and **pages**. When we instantiate two books, each object stores different information while still being part of the same class. This highlights the power of object-oriented programming in modelling complex systems.

Real-World Examples

Real-world examples show how object-oriented programming (OOP) concepts apply in practical scenarios, making it easier to understand how these concepts function in real applications.

Example 1: BankAccount

Class: BankAccount

Attributes: balance, account_holder

Methods: deposit(), withdraw()

Python

```
class BankAccount:
    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
        else:
            print("Insufficient funds")

account = BankAccount("Alice", 1000)
account.deposit(500)
account.withdraw(200)
print(account.balance) # Output: 1300
```

In this example, the `BankAccount` class models a bank account with attributes for the account holder's name and the account balance. Methods like `deposit` and `withdraw` allow us to manipulate the balance. The `account` object, representing Alice's account, is created

with an initial balance of \$1000, and further operations are performed on this object.

Example 2: Employee

Class: `Employee`

Attributes: `name`, `position`, `salary`

Methods: `promote()`, `give_raise()`

Python

```
class Employee:
    def __init__(self, name, position, salary):
        self.name = name
        self.position = position
        self.salary = salary

    def promote(self):
        self.position = "Senior " + self.position

    def give_raise(self, amount):
        self.salary += amount

emp = Employee("John", "Developer", 50000)
emp.promote()
emp.give_raise(5000)
print(emp.position, emp.salary) # Output: Senior Developer 55000
```

The `Employee` class models a company employee with a name, position, and salary. The methods allow promoting an employee to a higher position and giving a raise. In this case, the employee "John" is promoted to a senior developer and receives a salary increase, demonstrating how the object's attributes can be manipulated over time.

THANK YOU

